



BASES DE DONNÉES SQL

Dr. Cheikh Ibrahima Fall DIOP

✉: mamecheikhdiop@gmail.com

☎: (+221)7 $\left[\frac{6}{7}\right]$ 543 01 00

Table des matières

1	Le langage de définition des données	2
1.1	La commande CREATE	3
1.2	La commande ALTER	5
1.3	La commande DROP	5
1.4	La commande TRUNCATE	5
1.5	Les contraintes dans SQL	6
1.5.1	Contrainte NOT NULL	6
1.5.2	Contrainte DEFAULT	7
1.5.3	Contrainte UNIQUE	7
1.5.4	Contrainte CHECK	8
1.5.5	Contrainte PRIMARY KEY	9
1.5.6	Contrainte FOREIGN KEY	10
1.5.7	Contrainte INDEX	10
2	Le langage de manipulation des données	12
2.1	La commande INSERT	12
2.2	La commande DELETE	13
2.3	La commande UPDATE	13
3	Le langage de contrôle des données	15
3.1	La commande GRANT	15
3.2	La commande DENY	15
3.3	La commande REVOKE	16
3.4	La commande LOCK	16
4	Le langage de contrôle des transactions	17
4.1	La commande COMMIT	17
4.2	La commande ROLLBACK	18

4.3	La commande SAVEPOINT	18
5	Le langage d'interrogation des données	19
5.0.1	Trier les données - ORDER BY	35
6	Sous requêtes et Jointures	38
6.1	Les Sous requêtes	38
6.2	Les Jointures	42
7	Programmation dans SQL	49
7.1	Les procédures stockées	49
7.2	Les fonctions	52
7.3	Les triggers	55
8	Administration du serveur	58
9	Les transactions	59

Introduction

SQL (Structured Query Language) qu'on peut traduire en français "langage de requêtes structurées" est le langage standard utilisé le plus couramment aujourd'hui pour créer, modifier, lire, mettre à jour, ... des bases données relationnelles.

Il existe 5 type de commandes SQL : DDL (Data Definition Language), DML (Data Manipulation Language), DCL (Data Control Language), TCL (Transaction Control Language), et DQL (Data Query Language).

1

Le langage de définition des données

C'est le sous-ensemble de SQL qui permet de pour manipuler les structures de données d'une base de données, et non les données elles-mêmes.

Il permet :

1. de définir le domaine des données, c'est-à-dire l'ensemble des valeurs que peut prendre une donnée : nombre, chaîne de caractères, date, booléen.
2. de regrouper les données ayant un lien conceptuel au sein d'une même entité.
3. de définir les liens entre plusieurs entités de nature différente.
4. d'ajouter des contraintes de valeur sur les données.

On distingue typiquement trois types de commandes SQL de définition de données : Ces commandes peuvent porter sur les struc-

Commande	Description
CREATE	création d'une structure de données
ALTER	modification d'une structure de données
DROP	suppression d'une structure de données
TRUNCATE	vider le contenu d'une table

tures de données de type suivantes : DATABASE (base de données), TABLE table, INDEX (indice), VIEW (table virtuelle), SEQUENCE (suite de nombres), SYNONYM (synonyme) et USER (utilisateur).

1.1 La commande CREATE

Pour créer une base de données la commande est :

```
1 CREATE DATABASE mabase;
```

Alors que pour une table on devra écrire

```
1 CREATE TABLE nom_table (  
2     colone1 type,  
3     colone2 type,  
4     colone3 type,  
5     ....  
6 );
```

Chaque champs d'une table possède un type : CHAR, VARCHAR, INTEGER, NUMBER, DECIMAL, FLOAT, DOUBLE, DATE, TIME, TIMESTAMP, etc.

Ainsi, la création de la table serais :

```
1 CREATE TABLE Personne (  
2     ID integer,  
3     Nom varchar(25),  
4     Prenom varchar(50),  
5     Adresse varchar(255)  
6 );
```



Lors de la création d'une table on doit définir l'identifiant avec PRIMARY KEY. Il est aussi possible qu'une clé étrangère soit présente dans une table dans ce cas on devra le marquer par FOREIGN KEY.

```
1 CREATE TABLE Commande (  
2     IDCommande int NOT NULL,  
3     Montant int NOT NULL,
```

```

4 IDPersonne int,
5 PRIMARY KEY (IDCommande),
6 FOREIGN KEY (IDPersonne) REFERENCES Personne(ID)
7 );

```

Le langage SQL est normalisé, c'est-à-dire qu'un organisme s'est chargé d'établir la bonne façon de l'utiliser afin que le langage soit semblable d'un SGBD à l'autre.



Les différents SGBDR qui l'utilisent n'ont cependant pas suivi toutes les normes du standard SQL, ce qui fait que la syntaxe sera légèrement différente de l'un à l'autre. Les fonctionnalités supportées dépendent également du SGBDR.

Voici un exemple de requête SQL qui sera différente entre les SGBD :

Exemple :
MySQL

```

1 CREATE TABLE 'etudiants' (
2   'id' int(11) NOT NULL AUTO_INCREMENT,
3   'nomfamille' varchar(50) COLLATE utf8_unicode_ci,
4   'prenom' varchar(50) COLLATE utf8_unicode_ci,
5   'da' varchar(6) COLLATE utf8_unicode_ci,
6   PRIMARY KEY('id')
7 ) ENGINE=InnoDB DEFAULT CHARSET=utf8 COLLATE=utf8_unicode_ci
;

```

SQLite

```

1 CREATE TABLE IF NOT EXISTS 'etudiants' (
2   'id' INTEGER PRIMARY KEY AUTOINCREMENT,
3   'nomfamille' TEXT,
4   'prenom' TEXT,
5   'da' TEXT);

```

MS SQL

```
1 CREATE TABLE [dbo].[etudiants] (  
2 [id] INT IDENTITY(1, 1) NOT NULL,  
3 [nomfamille] NVARCHAR(50),  
4 [prenom] NVARCHAR(50),  
5 [da] NVARCAHR(6)  
6 );
```

1.2 La commande ALTER

Elle permet de modifier un champs d'une table, d'ajouter un nouveau champs ou d'en supprimer un.

```
1 ALTER TABLE Personne ADD (Telephone integer);  
2 ALTER TABLE Personne MODIFY Nom varchar(10);  
3 ALTER TABLE Personne DROP Nom ;
```

1.3 La commande DROP

```
1 DROP DATABASE nombase;  
2 DROP TABLE nomtable;
```

1.4 La commande TRUNCATE

Elle supprime toutes les données d'une table sans supprimer la table en elle-même. Elle diffère de la commande DROP qui supprime les données et que la table qui les contient.



TRUNCATE produit le même effet que l'instruction DELETE sans utilisation de WHERE.

1 `TRUNCATE TABLE ma_table ; -- purge la table ma table n`

1.5 Les contraintes dans SQL

Les contraintes sont les règles appliquées aux colonnes de données d'une table. Elles sont utilisées pour contrôler les informations pouvant être enregistrées dans une table en garantissant l'exactitude et la fiabilité des données de la base de données.

Les contraintes peuvent être au niveau de la colonne ou de la table. Les contraintes de niveau de colonne ne sont appliquées qu'à une seule colonne, alors que les contraintes de niveau de table s'appliquent à l'ensemble de la table.

Les contraintes les plus communes sont :

- ☐ NOT NULL
- ☐ DEFAULT
- ☐ UNIQUE
- ☐ CHECK
- ☐ PRIMARY KEY
- ☐ FOREIGN KEY
- ☐ INDEX

1.5.1 Contrainte NOT NULL

Par défaut, une colonne peut contenir des valeurs NULL. Si vous ne souhaitez pas qu'une colonne ait une valeur NULL, vous devez définir une telle contrainte sur cette colonne en spécifiant que NULL n'est plus autorisé pour cette colonne.

```
1 CREATE TABLE Employes(  
2     Id INT NOT NULL,  
3     Nom VARCHAR (20) NOT NULL,  
4     Age INT NOT NULL,  
5     Salaire DECIMAL (18, 2),  
6     PRIMARY KEY (Id)  
7 );
```

1.5.2 Contrainte DEFAULT

La contrainte DEFAULT fournit une valeur par défaut à une colonne lorsque l'instruction INSERT INTO ne fournit pas de valeur spécifique.

```
1 CREATE TABLE Employes(  
2     Id INT NOT NULL,  
3     Nom VARCHAR (20) NOT NULL,  
4     Age INT NOT NULL,  
5     Salaire DECIMAL (18, 2) DEFAULT 5000.00,  
6     PRIMARY KEY (Id)  
7 );
```

1.5.3 Contrainte UNIQUE

La contrainte UNIQUE empêche que deux enregistrements aient des valeurs identiques dans une colonne.

```
1 CREATE TABLE Employes(  
2     Id INT NOT NULL,  
3     Nom VARCHAR (20) NOT NULL UNIQUE,  
4     Age INT NOT NULL,  
5     Salaire DECIMAL (18, 2),  
6     PRIMARY KEY (Id)  
7 );
```

1.5.4 Contrainte CHECK

La contrainte **CHECK** active une condition permettant de vérifier la valeur saisie dans un enregistrement. Si la condition est évaluée à **false**, l'enregistrement viole la contrainte et n'est pas entré dans la table.

```
1 CREATE TABLE Employes(  
2     Id INT NOT NULL,  
3     Nom VARCHAR (20) NOT NULL,  
4     Age INT NOT NULL CHECK (Age >= 18),  
5     Salaire DECIMAL (18, 2),  
6     PRIMARY KEY (Id)  
7 );
```

La condition peut aussi porter sur une liste de valeurs autorisées.

```
1 CREATE TABLE Employes(  
2     Id INT NOT NULL,  
3     Nom VARCHAR (20) NOT NULL,  
4     Age INT NOT NULL CHECK (Age >= 18),  
5     Salaire DECIMAL (18, 2),  
6     Sexe VARCHAR(5) CHECK (Sexe IN ('Homme', 'Femme')),  
7     PRIMARY KEY (Id)  
8 );
```

Elle peut aussi être une liste de valeurs non autorisées et dans ce cas on utilise la clause **NOT IN**

Il est possible de nommer une contrainte CHECK (ou un contrainte portant sur une clé étrangère) au moment de sa création en utilisant la clause CONSTRAINT. Dans ce cas le nom défini est affiché lorsque la contrainte est violée.



```
1 CREATE TABLE Employes(  
2     Id INT NOT NULL,  
3     Nom VARCHAR (20) NOT NULL,  
4     Age INT NOT NULL CONSTRAINT Controle_Age  
        CHECK (Age >= 18),  
5     PRIMARY KEY (Id)  
6 );
```

En procédant de cette manière il est possible de supprimer la contrainte avec l'écriture suivante :

```
1 ALTER TABLE Employes DROP CHECK Controle_Age;
```

1.5.5 Contrainte PRIMARY KEY

Une clé primaire est un champ dans une table qui identifie de manière unique chaque ligne/enregistrement dans une table de base de données. Les clés primaires doivent contenir des valeurs uniques. Une colonne de clé primaire ne peut pas avoir de valeur NULL.

Une table ne peut avoir qu'une seule clé primaire, qui peut consister en un ou plusieurs champs. Lorsque plusieurs champs sont utilisés comme clé primaire, ils sont appelés clé composite.

Si une table a une clé primaire définie sur un ou plusieurs champs, vous ne pouvez pas avoir deux enregistrements ayant la même valeur pour ces champs.

```
1 CREATE TABLE Employes(  
2     Id INT NOT NULL,
```

```

3  Nom VARCHAR (20) NOT NULL,
4  Age INT NOT NULL,
5  Salaire DECIMAL (18, 2),
6  PRIMARY KEY (Id)
7 );

```

1.5.6 Contrainte FOREIGN KEY

Une clé étrangère est une clé utilisée pour relier deux tables. Ceci est parfois appelé aussi clé de référencement.

Une clé étrangère est une colonne ou une combinaison de colonnes dont les valeurs correspondent à une clé primaire dans une autre table.

La relation entre 2 tables correspond à la clé primaire dans l'une des tables avec une clé étrangère dans la seconde table.

```

1 CREATE TABLE Employes(
2     Id INT NOT NULL,
3     Nom VARCHAR (20) NOT NULL,
4     Age INT NOT NULL,
5     Salaire DECIMAL (18, 2),
6     PRIMARY KEY (Id)
7 );
8 CREATE TABLE Conges(
9     Code INT NOT NULL,
0     Date_debut DATE NOT NULL,
1     Date_fin DATE NOT NULL,
2     ID_EMP INT NOT NULL,
3     PRIMARY KEY (Code),
4     FOREIGN KEY (ID_EMP) REFERENCES Employes(Id)
5 );

```

1.5.7 Contrainte INDEX

Un index est utilisé pour créer et récupérer des données de la base de données très rapidement. Il peut être créé en utilisant une

seule colonne ou un groupe de colonnes dans une table. Lors de la création de l'index, un ROWID est attribué à chaque ligne avant le tri des données.

Pour créer un index sur la colonne Age, afin d'optimiser la recherche d'employés pour un âge spécifique, vous pouvez utiliser la syntaxe suivante :

```
1 CREATE INDEX idx_age  
2   ON Employes (Age);
```

2

Le langage de manipulation des données

Permet de manipuler les données d'une base de données. On distingue typiquement trois types de commandes SQL dédiées à la manipulation des données :

Commande	Description
INSERT	insertion de données dans une table
DELETE	suppression de données d'une table
UPDATE	mise à jour des données d'une table

2.1 La commande INSERT

Insertion de données dans une table :

```
1 INSERT INTO eleves (nom, prenom)
2 VALUES ('DIOUF', 'Moussa');
```

Si les données sont contenues dans un fichier .csv il est possible de les importer directement sous MySQL avec la clause `LOAD DATA INFILE` qui prend en compte les fichiers .txt en plus des fichiers .csv.



```
1 LOAD DATA INFILE 'fichier.csv' INTO TABLE
   matable FIELDS TERMINATED BY ';' ENCLOSED
   BY '"' LINES TERMINATED BY '\n' IGNORE 1
   LINES;
2 LOAD DATA LOCAL INFILE 'fichier.txt' INTO TABLE
   matable FIELDS TERMINATED BY "," LINES
   TERMINATED BY "\n" IGNORE 1 LINES;
```



Si l'exécution renvoie l'erreur suivante `ERROR 1290 (HY000): The MySQL server is running with the -secure-file-priv option so it cannot execute this statement`, il faut exécuter la commande `SHOW VARIABLES LIKE 'secure_file_priv'`; et placer le fichier dans le dossier renvoyé comme résultat.

2.2 La commande DELETE

Suppression de données dans une table :

```
1 DELETE FROM eleves
2 WHERE prenom = 'DIOP' and nom = 'Doudou';
```

2.3 La commande UPDATE

Mise à jour de données dans une table :

```
1 UPDATE eleves
2 SET prenom = 'Demba'
3 WHERE nom = 'SANE';
```

3

Le langage de contrôle des données

Est utilisé pour contrôler l'accès aux données d'une base de données. On distingue typiquement quatre types de commandes SQL pour le contrôle des données

Commande	Description
GRANT	autorisation d'un utilisateur à effectuer une action
DENY	interdiction à un utilisateur d'effectuer une action
REVOKE	annulation d'une commande de contrôle de données précédente
LOCK	verrouillage sur une structure de données

3.1 La commande GRANT

Autorisation d'un utilisateur à effectuer une action :

```
1 GRANT UPDATE (nom, prenom) ON eleves
2 TO enseignant
3 WITH GRANT OPTION;
```

3.2 La commande DENY

Interdiction d'un utilisateur à effectuer une action :

```
1 DENY DELETE
2 TO enseignant;
```

3.3 La commande REVOKE

Annulation d'une commande de contrôle de données précédente :

```
1 REVOKE UPDATE (nom, prenom)
2 ON eleves
3 FROM enseignant;
```

3.4 La commande LOCK

Verrouillage d'une structure de données :

```
1 LOCK TABLE eleves IN EXCLUSIVE MODE;
```

4

Le langage de contrôle des transactions

Il est utilisé pour le contrôle transactionnel dans une base de données, c'est-à-dire les caractéristiques des transactions, la validation et l'annulation des modifications.

Commande	Description
COMMIT	validation d'une transaction en cours
ROLLBACK	annulation d'une transaction en cours
SAVEPOINT	création d'un point de sauvegarde



Si ROLLBACK annule toute une transaction, SAVEPOINT permet de retourner jusqu'à une étape marquée comme point de sauvegarde sans annuler l'ensemble de la transaction

4.1 La commande COMMIT

Validation d'une transaction en cours :

```
1 START TRANSACTION;  
2 UPDATE eleves  
3 SET age = '19'  
4 WHERE nom = 'BA' and prenom = 'Adji';  
5 COMMIT;
```

4.2 La commande ROLLBACK

Annulation d'une transaction en cours :

```
1 DELETE FROM eleves
2 WHERE age = 25;
3 ROLLBACK;
```

4.3 La commande SAVEPOINT

Annulation d'une jusqu'à une sauvegarde :

```
1 DELETE FROM eleves
2 WHERE age = 25;
3 SAVEPOINT sauvegarde
4 DELETE FROM eleves
5 WHERE nom = 'DIOP';
6 ROLLBACK TO sauvegarde;
```

5

Le langage d'interrogation des données

Est principalement ce à quoi on pense lorsqu'on parle de SQL. Il est matérialisé par la commande `SELECT` qui permet de faire sélection de données dans une table.

Il possède des caractéristiques proches de l'algèbre relationnelle (jointure par emboîtement) et d'autres proches du calcul des tuples (variables sur les relations) et utilise des opérateurs arithmétiques, de comparaison et logiques.

Les opérateurs arithmétiques :

Opérateur	Description
+	Ajoute des valeurs de chaque côté de l'opérateur.
-	Soustrait l'opérande droit de l'opérande gauche.
*	Multiplie les valeurs de chaque côté de l'opérateur.
/	Divise l'opérande gauche par l'opérande droit.
%	Divise l'opérande gauche par l'opérande droit et renvoie le reste.

Les opérateurs de comparaison :

Opérateur	Description
=	Vérifie si les valeurs de deux opérandes sont égales ou non, si oui, la condition devient vraie.

Opérateur	Description
$! =$	Vérifie si les valeurs de deux opérandes sont égales ou non, si les valeurs ne sont pas égales, alors la condition devient vraie.
$< >$	Vérifie si les valeurs de deux opérandes sont égales ou non, si les valeurs ne sont pas égales, alors la condition devient vraie.
$>$	Vérifie si la valeur de l'opérande gauche est supérieure à la valeur de l'opérande droit. Si oui, la condition devient vraie.
$<$	Vérifie si la valeur de l'opérande gauche est inférieure à la valeur de l'opérande droit. Si oui, la condition devient vraie.
$> =$	Vérifie si la valeur de l'opérande de gauche est supérieure ou égale à la valeur de l'opérande de droite. Si oui, la condition devient vraie.
$< =$	Vérifie si la valeur de l'opérande gauche est inférieure ou égale à la valeur de l'opérande droit. Si oui, la condition devient vraie.
$! <$	Vérifie si la valeur de l'opérande gauche n'est pas inférieure à la valeur de l'opérande droit. Si la réponse est oui, la condition devient vraie.
$! >$	Vérifie si la valeur de l'opérande de gauche n'est pas supérieure à la valeur de l'opérande de droite, si oui, la condition devient vraie.

Les opérateurs logiques :

Opérateur	Description
ALL	compare une valeur à toutes les valeurs d'un autre jeu de valeurs.
ANY	compare une valeur à une valeur applicable de la liste conformément à la condition.
BETWEEN	recherche des valeurs comprises dans un ensemble de valeurs, en fonction de la valeur minimale et de la valeur maximale.
EXISTS	recherche la présence d'une ligne dans une table spécifiée qui répond à un certain critère.
IN	compare une valeur à une liste de valeurs littérales spécifiées.
LIKE	compare une valeur à des valeurs similaires à l'aide d'opérateurs génériques.
NOT	inverse la signification de l'opérateur logique avec lequel il est utilisé. Ex.: NOT EXISTS, NOT BETWEEN, NOT IN, etc.
AND	permet l'existence de plusieurs conditions dans la clause WHERE d'une instruction SQL.
OR	combine plusieurs conditions dans la clause WHERE d'une instruction SQL.
IS NULL	comparer une valeur avec une valeur NULL.
UNIQUE	recherche dans chaque ligne de la table spécifiée l'unicité (pas de doublons).

Les exemples dans cette partie s'appuient sur la base de données relative aux fournisseurs (F), produits (P), usines (U) et livraisons (PUF), décrite par le schéma suivant:

F (NF, nomF, statut, ville) P (NP, nomP, poids, couleur) U (NU,

nomU, ville) PUF (NP, NU, NF, qt)

Format de base d'une requête

Le format général du **SELECT** est le suivant :
Soit une relation R (A1, A2, ... , An)

```
1 SELECT [Ai1] [,Ai2] ... [,Aiu] [,f1 (Aj1)] [,f2 (Aj2)] ...  
    [,fv (Ajv)]  
2 FROM A1, A2, ... , An  
3 [WHERE <condition1 portant sur chaque tuple de R>]  
4 [GROUP BY Ak1 [,Ak2] ... [,Akw]]  
5 [HAVING <condition2 portant sur chaque groupe de tuples de R  
    >]  
6 [ORDER BY Ak1 [,Ak2] ... [,Akw]]  
7 LIMIT nombre
```

avec fi = fonction d'agrégation (**COUNT**, **SUM**, **MIN**, **MAX**, **AVG**) ,
et $Ak1, Ak2, \dots, Akw \supseteq Ai1, Ai2, \dots, Aiu$,
et $Ak1, Ak2, \dots, Akw \cap Aj1, Aj2, \dots, Ajv = \emptyset$.
Exemple: nom et poids des produits rouges.

```
1 SELECT nomP, poids FROM P  
2 WHERE couleur = "rouge";
```

Exemple : tous les renseignements sur tous les fournisseurs.

```
1 SELECT NF, nomF, statut, ville FROM F;
```

ou

```
1 SELECT * FROM F;
```

Un résultat sans doubles :

Les SGBD commercialisés (dont les SQL...) ne suppriment pas

automatiquement les doubles. La clause **DISTINCT** permet à l'utilisateur d'avoir un résultat sans double.

Exemple : liste des couleurs qui existent.

```
1 SELECT DISTINCT couleur FROM P;
```

Un résultat trié :

La clause **ORDER BY** permet de définir un ordre de tri pour les tuples du résultat.

Exemple : liste des fournisseurs de Pikine par ordre alphabétique.

```
1 SELECT nomF, NF, statut
2 FROM F
3 WHERE ville = "Pikine" ORDER BY nomF DESC , NF ASC;
```

Recherche avec l'opérateur logique IN :

IN vérifie si un résultat correspond à un ensemble de valeurs et remplace une succession de OR

Exemple : noms des fournisseurs numéro 1, 2, 3.

```
1 SELECT nomF FROM F
2 WHERE NF = 1 OR NF =2 OR NF = 3;
```

ou

```
1 SELECT nomF FROM F
2 WHERE NF IN (1, 2, 3);
```

Recherche dans un intervalle avec BETWEEN :

```
1 SELECT nomF FROM F
2 WHERE NF BETWEEN 1 AND 3;
```

Fonctions d'agrégation

SQL offre des fonctions d'agrégation pour effectuer des calculs sur plusieurs lignes d'une seule colonne d'une table. Elles retournent une valeur unique. Elles sont également utilisées pour résumer les données; ainsi, cardinal : **COUNT** moyenne : **AVG** minimum et maximum : **MIN**, **MAX** et total : **SUM** opèrent sur un ensemble de valeurs prises par un attribut alors que **COUNT** peut concerner sur un ensemble de tuples.



Toutes les fonctions d'agrégation excluent par défaut les valeurs NULL avant de travailler sur les données.

Considérons la table échantillon - Employes suivante :

Id	Nom	Age	Salaire	Profession	Dep
1	Ismail	25	6000.00	Assistant	2
2	Mohamed	30	8000.40	Directeur	1
3	Fatima	29	6000.00	Directeur	3
4	Dounia	30	7000.00	Assistant	4
5	Omar	29	9000.00	Ingenieur	1
6	Mostafa	29	7500.00	Ingenieur	NULL

COUNT

La fonction COUNT est utilisée pour compter le nombre de lignes dans une table de base de données. Il peut fonctionner sur les types de données numériques et non numériques.

La fonction COUNT utilise COUNT(*) qui renvoie le nombre

de toutes les lignes d'une table spécifiée. COUNT(*) considère les doublons et Null.

Syntaxe :

```
1 COUNT(*)
2 // ou
3 COUNT([ALL|DISTINCT] nom_colonne)
```

Exemple 1 : La requête suivante comptera les enregistrements dans la table Employees

```
1 SELECT count(*) FROM Employees;
```

Cette requête produira le jeu de résultats suivant :

```
+-----+
| count(*) |
+-----+
|          6 |
+-----+
```

Exemple 2 : Compter les employés affectés à un département

```
1 SELECT count(Dep) FROM Employees;
```

```
+-----+
| count(Dep) |
+-----+
|           5 |
+-----+
```

Exemple 3 : Compter les différents âges de la table Employees, (sans doublons)

```
1 SELECT count(DISTINCT Age) FROM Employes;
```

```
+-----+
| count(DISTINCT Age) |
+-----+
|                    3 |
+-----+
```

SUM

La fonction SUM renvoie la somme de toutes les valeurs de la colonne spécifiée. SUM fonctionne uniquement sur les champs numériques.

Syntaxe :

```
1 SUM([ALL|DISTINCT] nom_colonne)
```

Exemple 4 : La requête suivante renvoie la somme des salaires

```
1 SELECT SUM(Salaire) FROM Employes;
```

```
+-----+
| SUM(Salaire) |
+-----+
|      43500.40 |
+-----+
```

Exemple 5 : La requête suivante renvoie la somme des âges sans compter les valeurs dupliquées

```
1 SELECT SUM(DISTINCT Age) FROM Employes;
```

```
+-----+
| SUM(DISTINCT Age) |
+-----+
|                  84 |
+-----+
```

AVG

La fonction AVG renvoie la moyenne des valeurs d'une colonne spécifiée. Tout comme la fonction SUM, elle ne fonctionne que sur les types de données numériques.

Syntaxe :

```
1  AVG([ALL|DISTINCT] nom_colonne)
```

Exemple 6 : La requête suivante renvoie le salaire moyen de la table Employes

```
1  SELECT AVG(Salaire) FROM Employes;
```

```
+-----+
| AVG(Salaire) |
+-----+
| 7250.066667 |
+-----+
```

MIN

La fonction MIN est utilisée pour déterminer la plus petite valeur de toutes les valeurs sélectionnées d'une colonne.

Syntaxe :

```
1  MIN([ALL|DISTINCT] nom_colonne)
```

Exemple 7 : La requête suivante renvoie le salaire minimum de la table Employes

```
1 SELECT MIN(Salaire) FROM Employes;
```

```
+-----+
| MIN(Salaire) |
+-----+
|      6000.00 |
+-----+
```

MAX

Comme son nom l'indique, la fonction MAX est l'opposé de la fonction MIN. Elle renvoie la plus grande valeur de toutes les valeurs sélectionnées d'une colonne.

Syntaxe :

```
1 MAX([ALL|DISTINCT] nom_colonne)
```

Exemple 8 : La requête suivante renvoie le salaire maximum de la table Employes

```
1 SELECT MAX(Salaire) FROM Employes;
```

```
+-----+
| MAX(Salaire) |
+-----+
|      9000.00 |
+-----+
```

Recherche avec partition des tuples d'une relation

Clause **GROUP BY**

Exemple: combien de produits différents ont été livrés par chacun des fournisseurs ?

Il faut partitionner l'ensemble des tuples de PUF en un sous-ensemble (ou groupe) par numéro de fournisseur. C'est ce que permet la clause **GROUP BY**.

```
1 SELECT NF, COUNT (DISTINCT NP)
2 FROM PUF
3 GROUP BY NF;
```

Cette instruction génère dans le SGBD les actions suivantes :

1. Classer les tuples de PUF, groupe par groupe (un groupe = ensemble des tuples ayant même NF),
2. Pour chaque groupe évaluer le résultat du **SELECT** (compter le nombre de NP différents dans ce groupe).

Clause **HAVING <condition>**

Exemple: combien de produits différents ont été livrés par chacun des fournisseurs, tels que la quantité totale de produits livrés par ce fournisseur soit supérieure à 1000 ?

```
1 SELECT NF, COUNT (DISTINCT NP) FROM PUF
2 GROUP BY NF
3 HAVING SUM (qt) > 1000;
```



La clause "**HAVING** <condition>" permet de sélectionner les groupes qui satisfont une condition. Attention, contrairement à la clause "**WHERE** <condition>", la condition ne porte pas sur un tuple mais sur l'ensemble des tuples d'un groupe.

La condition du **HAVING** peut être de deux types :

1. Condition comparant le résultat d'une fonction d'agrégation portant sur un attribut qui ne fait pas partie de la clause **GROUP BY**:
 $f(A_{jx})$ <opérateur de comparaison> valeur
Cette fonction porte alors sur l'ensemble des valeurs prises par l'attribut pour le groupe de tuples;
2. Condition comparant l'ensemble des valeurs prises par un attribut (qui ne fait pas partie de la clause **GROUP BY**) pour les tuples du groupe; cette comparaison se fait en général par rapport à un ensemble, à l'aide des comparateurs logiques d'ensembles:
 $=, \neq, \text{CONTAINS}, \text{NOT CONTAINS}.$

Dans ce dernier cas, afin d'exprimer le fait qu'il s'agit de l'ensemble des valeurs prises par l'attribut pour tous les tuples du groupe, le nom de l'attribut est précédé du mot clé **SET**.

La condition du **HAVING** peut donc s'écrire :

SET nom-attribut <opérateur de comparaison ensembliste> <ensemble> avec <opérateur de comparaison ensembliste> ::= = $\neq$
CONTAINS NOT CONTAINS

L'instruction ci-dessus génère dans le SGBD les actions suivantes :

1. Créer une relation de travail, R' , qui est une copie de R ; éliminer de R' les tuples qui ne satisfont pas la condition du **WHERE**.

2. Classer les tuples de R', groupe par groupe (un groupe = ensemble des tuples ayant même Ak1 [,Ak2] ... [,Akw]).
3. Pour chaque groupe de tuples de R' faire:
 - tester la condition du **HAVING**
 - si elle est vérifiée, alors évaluer le résultat du **SELECT**.

Exemple: numéros des fournisseurs qui fournissent au moins tous les produits fournis par le fournisseur numéro 100.

```
1 SELECT NF
2 FROM PUF GROUP BY NF
3 HAVING SET NP /* ensemble des produits du fournisseur NF*/
4 CONTAINS
5 (SELECT NP FROM PUF WHERE NF=100);
```

Recherche sur plusieurs relations simultanément

Format général:

```
SELECT Ai...
FROM R1, R2..., Rn
WHERE condition de jointure entre les Ri
AND condition(s) de la requête
```

Exemple : pour chaque produit livré, le nom du produit et les villes de leurs fournisseurs.

```
1 SELECT nomP, ville FROM P, F, PUF
2 WHERE PUF.NP = P.NP AND PUF.NF = F.NF;
```

Modèles de recherche - LIKE

Cette partie est tirée du cours du Pr Mostafa ESSADDOUKI¹ qui a donnée son accord pour que je les utilise. Un grand merci à lui.

1. <https://developpement-informatique.com/tutoriel/19/langage-sql---guide-complet-pour-les-debutants>

Supposons que nous souhaitons extraire les colonnes où les n-uplets commencent et/ ou se terminent par une lettre ou une chaîne de caractère. Nous utilisons alors la clause **LIKE** qui est souvent associée à la clause **WHERE** en **SQL**.

Deux types de caractères génériques sont utilisés pour filtrer les résultats:

1. **%** : Utilisé pour faire correspondre zéro, un ou plusieurs caractères. (Longueur variable)
2. **_** : Utilisé pour correspondre exactement à un caractère. (Longueur fixe)

Voici les règles utilisées pour la correspondance de modèle avec la clause **LIKE**:

Modèle	Description
"a%"	Il fait correspondre les chaînes qui commencent par "a"
"%a"	Il fait correspondre les chaînes qui se terminent par "a"
"a%t"	Il fait correspondre les chaînes qui commencent par «a» et se terminent par «t».
"%abc%"	Il fait correspondre les chaînes qui contiennent la sous-chaîne "abc" en n'importe quelle position.
"_abc%"	Il fait correspondre les chaînes contenant la sous-chaîne "abc" en deuxième position.
"_a%"	Il fait correspondre les chaînes contenant «a» à la deuxième position.
"a_%_%"	Il fait correspondre les chaînes qui commencent par "a" et contiennent au moins 2 caractères supplémentaires.

Exemples

Soit la table échantillon - Employes

Id	Nom	Age	Salaire
1	Ismail	25	6000.00
2	Mohamed	30	8000.40
3	Fatima	29	6000.00
4	Dounia	31	9000.00
5	Omar	30	7500.00
6	Mostafa	32	7000.00

Les employés dont le nom commence par "Mo"

```
1 SELECT * FROM Employes WHERE Nom LIKE "Mo%";
```

Id	Nom	Age	Salaire
2	Mohamed	30	8000.40
6	Mostafa	32	7000.00

Les employés dont le nom se termine par "a"

```
1 SELECT * FROM Employes WHERE Nom LIKE "%a";
```

+	----	+	-----	+	----	+	-----	+
	Id		Nom		Age		Salaire	
+	----	+	-----	+	----	+	-----	+
	3		Fatima		29		6000.00	
	4		Dounia		31		9000.00	
	6		Mostafa		32		7000.00	
+	----	+	-----	+	----	+	-----	+

Les employés dont le nom contient un "m"

```
1 SELECT * FROM Employes WHERE Nom LIKE "%m%";
```

+	----	+	-----	+	----	+	-----	+
	Id		Nom		Age		Salaire	
+	----	+	-----	+	----	+	-----	+
	1		Ismail		25		6000.00	
	2		Mohamed		30		8000.40	
	3		Fatima		29		6000.00	
	5		Omar		30		7500.00	
	6		Mostafa		32		7000.00	
+	----	+	-----	+	----	+	-----	+

Les employés dont le nom se termine par un "m" suivi d'un autre caractere

```
1 SELECT * FROM Employes WHERE Nom LIKE "%m_";
```

Id	Nom	Age	Salaire
3	Fatima	29	6000.00

5.0.1 Trier les données - ORDER BY

L'instruction ORDER BY permet de trier le résultat d'une requête par ordre croissant ou décroissant selon une ou plusieurs colonnes.

Par défaut, ORDER BY trie les données par ordre croissant. Le mot-clé DESC permet de trier par ordre décroissant là le mot-clé ASC pour trier par ordre croissant.

```

1 SELECT liste-colonnes
2 FROM nom_table
3 [WHERE condition]
4 [ORDER BY colonne1, colonne2, .. ] [ASC | DESC];

```



Il est possible d'utiliser plusieurs colonnes dans la clause ORDER BY mais il faut s'assurer qu'elle figurent dans la liste des colonnes (liste-colonnes).

Lorsque ORDER BY est appliqué sur plusieurs colonnes, le tri commence par la première colonne, si deux ou plusieurs enregistrements ont le même rang, alors le tri passe à la colonne suivante, etc.

Exemples

Considérons toujours la table échantillon - Employes
Les employés par ordre croissant d'âge

```

1 SELECT * FROM Employes ORDER BY Age;

```

Id	Nom	Age	Salaire
1	Ismail	25	6000.00
4	Dounia	25	9000.00
3	Fatima	29	6000.00
2	Mohamed	30	8000.40
5	Omar	30	7500.00
6	Mostafa	32	7000.00

Les employés par ordre décroissant d'âge

```
1 SELECT * FROM Employes ORDER BY Age DESC;
```

Id	Nom	Age	Salaire
6	Mostafa	32	7000.00
2	Mohamed	30	8000.40
5	Omar	30	7500.00
3	Fatima	29	6000.00
1	Ismail	25	6000.00
4	Dounia	25	9000.00

Les employés par ordre croissant d'âge et décroissant de Nom

```
1 SELECT * FROM Employes ORDER BY Age, Nom DESC;
```

Id	Nom	Age	Salaire
1	Ismail	25	6000.00
4	Dounia	25	9000.00
3	Fatima	29	6000.00
5	Omar	30	7500.00
2	Mohamed	30	8000.40
6	Mostafa	32	7000.00

Les employés par ordre croissant d'âge et de Nom

```
SELECT * FROM Employes ORDER BY Age DESC, Nom ASC;
```

Id	Nom	Age	Salaire
6	Mostafa	32	7000.00
2	Mohamed	30	8000.40
5	Omar	30	7500.00
3	Fatima	29	6000.00
4	Dounia	25	9000.00
1	Ismail	25	6000.00

6

Sous requêtes et Jointures

6.1 Les Sous requêtes

Une sous-requête (appelée aussi "requête imbriquée" ou "requête en cascade") consiste à exécuter une requête à l'intérieur d'une autre requête. Elles est souvent utilisée au sein d'une clause **WHERE** ou de **HAVING** pour remplacer une ou plusieurs constante. Exemple : numéros des fournisseurs de produits rouges ? ensemble des numéros des produits rouges :

```
1 SELECT NP
2 FROM P
3 WHERE couleur = "rouge";
```

Ensemble des numéros des fournisseurs de produits rouges :

```
1 SELECT NF
2 FROM PUF
3 WHERE NP IN
4     (SELECT NP FROM P
5      WHERE couleur = "rouge");
```

Le mot clef **IN** signifie "appartient" dans le sens de l'opérateur mathématique de la théorie des ensembles ($\in$) La phrase **NP IN (SELECT NP FROM P WHERE couleur = "rouge")** est une condition logique, signifiant "la valeur de NP est dans l'ensemble des numéros de produits rouges", ce qui est vrai ou faux.

Pour répondre à cette requête, le SGBD :

1. exécute la requête interne (calcul de l'ensemble des numéros des produits rouges),

2. exécute la requête externe **SELECT NF FROM PUF WHERE NP IN (...)** en balayant PUF et en testant pour chaque tuple si ce NP appartient à l'ensemble des numéros de produits rouges.

Recherche avec quantificateurs : **SOME**, **ANY**, **ALL**

SQL permet d'écrire des conditions où apparaissent des quantificateurs proches de ceux de la logique ("il existe" ($\exists$), "quelque soit" ($\forall$)), grâce aux mots clefs **SOME**, **ANY** et **ALL**. Les mots clefs **SOME** et **ANY** ont exactement la même signification; ce sont des synonymes.

Le format général d'une condition élémentaire avec quantificateur est le suivant: <valeur attribut> <opérateur de comparaison> <quantificateur> <ensemble> avec <quantificateur> ::= **SOME ANY ALL** ce qui signifie:

pour **SOME** et **ANY** : "existe-t-il dans l'ensemble au moins un élément e qui satisfait la condition: <opérateur de comparaison> <ensemble> ?"

pour **ALL** : "tous les éléments de l'ensemble satisfont-ils la condition ?"

Le mot clef **IN** est équivalent à un quantificateur existentiel (**SOME** ou **ANY**) avec l'opérateur de comparaison d'égalité. **SOME** et **ANY** sont donc plus puissants. De même, les requêtes avec un quantificateur universel (**ALL**) et un comparateur d'égalité peuvent s'écrire avec une condition ensembliste (voir le paragraphe suivant). Cependant le mot clef **ALL** ne permet pas d'exprimer toutes les requêtes contenant un quantificateur du type "quelque soit".

On peut alors écrire la requête inverse avec un "NOT EXISTS" (voir paragraphe plus loin). Par exemple la requête "chercher les X qui pour tout Y satisfont telle condition" peut aussi s'exprimer: "chercher les X tels qu'il n'existe aucun Y qui ne satisfait pas telle condition".

Exemples:

Ensemble des numéros des fournisseurs de produits rouges :

```
1 SELECT NF
2 FROM PUF WHERE NP = ANY
3     (SELECT NP FROM P
4      WHERE couleur = "rouge");
```

Ensemble des numéros des fournisseurs qui ne fournissent que des produits rouges :

```
1 SELECT NF
2 FROM F
3 WHERE "rouge" = ALL
4     (SELECT couleur FROM P
5      WHERE NP = ANY (SELECT NP FROM PUF
6      WHERE PUF.NF = F.NF));
```

Recherche avec des conditions sur des ensembles

Dans les paragraphes précédents, les conditions élémentaires portant sur une valeur d'attribut ont été définies. D'autres types de conditions élémentaires permettent de comparer des ensembles entre eux. Ce sont:

Test d'égalité d'ensembles:

$\langle \text{ensemble 1} \rangle = \langle \text{ensemble 2} \rangle$

$\langle \text{ensemble 1} \rangle \neq \langle \text{ensemble 2} \rangle$

Test d'inclusion d'ensembles:

$\langle \text{ensemble 1} \rangle$ **CONTAINS** $\langle \text{ensemble 2} \rangle$

Cette condition signifie que l'ensemble 1 contient (ou est égal à) à l'ensemble 2, ce qui en théorie des ensembles s'écrirait : $\langle \text{ensemble 1} \rangle \supseteq \langle \text{ensemble 2} \rangle$.

La condition: $\langle \text{ensemble 1} \rangle$ **NOT CONTAINS** $\langle \text{ensemble 2} \rangle$ est la négation de la précédente; elle est vraie si un élément de l'ensemble 2 n'appartient pas à l'ensemble 1.

Exemple : noms des fournisseurs qui fournissent tous les produits rouges.

```
1 SELECT nomF
2 FROM F
3 WHERE (SELECT NP
4        FROM PUF/* ensemble des produits du fournisseur F*/
5        WHERE NF = F. NF)
6 CONTAINS
7 (SELECT NP
8        FROM P/* ensemble des produits rouges*/
9        WHERE couleur ="rouge");
```

EXISTS < ensemble >

Cette condition teste si l'ensemble n'est pas vide (ensemble $\neq \emptyset$).

Exemple : noms des fournisseurs qui fournissent au moins un produit rouge.

```
1 SELECT nom F FROM F
2 WHERE EXISTS (SELECT *
```

```
3 FROM PUF, P
4 WHERE NF = F.NF AND couleur ="rouge" AND PUF.NP=P.NP);
```

Il existe aussi la condition inverse : **NOT EXISTS** <ensemble> qui teste si l'ensemble est vide.

6.2 Les Jointures

La clause JOIN est utilisée pour récupérer les données de deux ou plusieurs tables, qui sont jointes pour apparaître comme un seul ensemble de données. Elle permet de combiner les colonnes de deux tables ou plus en utilisant des valeurs qui leur sont communes.

Le mot-clé JOIN est utilisé dans les requêtes SQL pour joindre deux tables ou plus. Les conditions minimales requises pour joindre la table sont (n-1), n étant le nombre de tables. Une table peut également se joindre à elle-même, appelée SELF JOIN.

Voici les types de jointure que nous pouvons utiliser en SQL:

- ✓ CROSS
- ✓ INNER
- ✓ LEFT
- ✓ RIGHT
- ✓ SELF

Pour cette partie considérons les deux tables suivantes :

1. Table - Employes

Id	Nom	Age	Salaire	Profession	Dep
1	Ismail	25	6000.00	Assistant	2
2	Mohamed	30	8000.40	Directeur	1
3	Fatima	29	6000.00	Directeur	3
4	Dounia	30	7000.00	Assistant	4
5	Omar	29	9000.00	Ingenieur	1
7	Mostafa	29	9000.00	Ingenieur	NULL

2. Table - Departement

Id_dep	Nom_dep
1	Informatique
2	RH
3	Vente
4	Strategies

CROSS JOIN

Ce type de JOIN renvoie le produit cartésien des lignes des tables de la jointure. Elle renverra un jeu de résultats des enregistrements combinant chaque ligne de la première table avec chaque ligne de la deuxième table.

Syntaxe:

```
1 SELECT liste-colonnes
2 FROM
3 table1 CROSS JOIN table2;
```

Exemple 1 :

```
1 SELECT * FROM Departement CROSS JOIN Employes;
```

Id_dep	Nom_dep	Id	Nom	Age	Salaire	Profession	Dep
1	Informatique	1	Ismail	25	6000.00	Assistant	2
2	RH	1	Ismail	25	6000.00	Assistant	2
3	Vente	1	Ismail	25	6000.00	Assistant	2
4	Strategies	1	Ismail	25	6000.00	Assistant	2
1	Informatique	2	Mohamed	30	8000.40	Directeur	1
2	RH	2	Mohamed	30	8000.40	Directeur	1
3	Vente	2	Mohamed	30	8000.40	Directeur	1
4	Strategies	2	Mohamed	30	8000.40	Directeur	1
1	Informatique	3	Fatima	29	6000.00	Directeur	3
2	RH	3	Fatima	29	6000.00	Directeur	3
3	Vente	3	Fatima	29	6000.00	Directeur	3
4	Strategies	3	Fatima	29	6000.00	Directeur	3
1	Informatique	4	Dounia	30	7000.00	Assistant	4
2	RH	4	Dounia	30	7000.00	Assistant	4
3	Vente	4	Dounia	30	7000.00	Assistant	4
4	Strategies	4	Dounia	30	7000.00	Assistant	4
1	Informatique	5	Omar	29	9000.00	Ingenieur	1
2	RH	5	Omar	29	9000.00	Ingenieur	1
3	Vente	5	Omar	29	9000.00	Ingenieur	1
4	Strategies	5	Omar	29	9000.00	Ingenieur	1
1	Informatique	7	Mostafa	29	9000.00	Ingenieur	NULL
2	RH	7	Mostafa	29	9000.00	Ingenieur	NULL
3	Vente	7	Mostafa	29	9000.00	Ingenieur	NULL
4	Strategies	7	Mostafa	29	9000.00	Ingenieur	NULL

Comme vous pouvez le constater, cette jointure renvoie le produit cartésien de tous les enregistrements présents dans les deux tables.

INNER JOIN

La jointure la plus importante et la plus utilisée est la jointure INNER. Elle est également appelée jointure d'égalité.

INNER JOIN crée un jeu de résultats en combinant les valeurs de colonne de deux tables (table1 et table2) en fonction du prédicat de jointure. La requête compare chaque ligne de table1(A) avec chaque ligne de table2(B) pour rechercher toutes les paires de lignes satisfaisant le prédicat de jointure. Lorsque le prédicat de jointure est satisfait, les valeurs de colonne de chaque paire de lignes correspondante de A et de B sont combinées dans une ligne de résultat.

Syntaxe:

```

1 SELECT liste-colonnes
2 FROM
3 table1 INNER JOIN table2
4 ON table1.champ_commun = table1.champ_commun;

```

La plupart des gens utilisent cette syntaxe à la place de la première syntaxe

```

1 SELECT liste-colonnes
2 FROM
3 table1, table2
4 WHERE table1.champ_commun = table1.champ_commun;

```

Exemple 2 :

```

1 SELECT * FROM Departement AS D INNER JOIN Employes AS E ON D
   .Id_dep=E.Dep;

```

Id_dep	Nom_dep	Id	Nom	Age	Salaire	Profession	Dep
2	RH	1	Ismail	25	6000.00	Assistant	2
1	Informatique	2	Mohamed	30	8000.40	Directeur	1
3	Vente	3	Fatima	29	6000.00	Directeur	3
4	Strategies	4	Dounia	30	7000.00	Assistant	4
1	Informatique	5	Omar	29	9000.00	Ingenieur	1

LEFT JOIN

LEFT JOIN renvoie toutes les lignes de la table de gauche, même s'il n'y a pas de correspondance dans la table de droite. Cela signifie que si la clause ON correspond à 0 (zéro) enregistrements dans la table de droite; la jointure retournera toujours une ligne dans le résultat, mais avec NULL dans chaque colonne de la table de droite.

Cela signifie qu'une jointure gauche renvoie toutes les valeurs de la table de gauche, ainsi que les valeurs correspondantes de la

table de droite ou NULL en cas d'absence de prédicat de jointure correspondant.

Syntaxe:

```
1 SELECT liste-colonnes
2 FROM
3 table1 LEFT JOIN table2
4 ON table1.champ_commun = table1.champ_commun;
```

Exemple 3 :

```
1 SELECT * FROM Employes AS E LEFT JOIN Departement as D ON D.
   Id_dep=E.Dep;
```

	Id	Nom	Age	Salaire	Profession	Dep	Id_dep	Nom_dep
	2	Mohamed	30	8000.40	Directeur	1	1	Informatique
	5	Omar	29	9000.00	Ingenieur	1	1	Informatique
	1	Ismail	25	6000.00	Assistant	2	2	RH
	3	Fatima	29	6000.00	Directeur	3	3	Vente
	4	Dounia	30	7000.00	Assistant	4	4	Strategies
	7	Mostafa	29	9000.00	Ingenieur	NULL	NULL	NULL

RIGHT JOIN

RIGHT JOIN renvoie toutes les lignes de la table de droite, même s'il n'y a pas de correspondance dans la table de gauche. Cela signifie que si la clause ON correspond à 0 (zéro) enregistrements dans la table de gauche; la jointure retournera toujours une ligne dans le résultat, mais avec NULL dans chaque colonne de la table de gauche.

Syntaxe:

```
1 SELECT liste-colonnes
2 FROM
3 table1 RIGHT JOIN table2
4 ON table1.champ_commun = table1.champ_commun;
```

Exemple 4 :

```
1 SELECT * FROM Employes AS E RIGHT JOIN Departement as D ON D
   .Id_dep=E.Dep;
```

Id	Nom	Age	Salaire	Profession	Dep	Id_dep	Nom_dep
1	Ismail	25	6000.00	Assistant	2	2	RH
2	Mohamed	30	8000.40	Directeur	1	1	Informatique
3	Fatima	29	6000.00	Directeur	3	3	Vente
4	Dounia	30	7000.00	Assistant	4	4	Strategies
5	Omar	29	9000.00	Ingenieur	1	1	Informatique

SELF JOIN

SELF JOIN est utilisée pour joindre une table à elle-même comme si la table était deux tables; renommer temporairement au moins une table dans l'instruction SQL.

Syntaxe:

```
1 SELECT liste-colonnes
2 FROM
3 table1 AS T1, table1 AS T2
4 WHERE T1.champ_commun = T2.champ_commun;
```

Ici, la clause WHERE peut être toute expression donnée en fonction de vos besoins.

Exemple 5 :

```
1 SELECT * FROM Employes AS T1, Employes AS T2 WHERE T1.
   Salaire>T2.Salaire;
```

Id	Nom	Age	Salaire	Profession	Dep	Id	Nom	Age	Salaire	Profession	Dep
2	Mohamed	30	8000.40	Directeur	1	1	Ismail	25	6000.00	Assistant	2
4	Dounia	30	7000.00	Assistant	4	1	Ismail	25	6000.00	Assistant	2
5	Omar	29	9000.00	Ingenieur	1	1	Ismail	25	6000.00	Assistant	2
7	Mostafa	29	9000.00	Ingenieur	NULL	1	Ismail	25	6000.00	Assistant	2
5	Omar	29	9000.00	Ingenieur	1	2	Mohamed	30	8000.40	Directeur	1
7	Mostafa	29	9000.00	Ingenieur	NULL	2	Mohamed	30	8000.40	Directeur	1
2	Mohamed	30	8000.40	Directeur	1	3	Fatima	29	6000.00	Directeur	3
4	Dounia	30	7000.00	Assistant	4	3	Fatima	29	6000.00	Directeur	3
5	Omar	29	9000.00	Ingenieur	1	3	Fatima	29	6000.00	Directeur	3
7	Mostafa	29	9000.00	Ingenieur	NULL	3	Fatima	29	6000.00	Directeur	3
2	Mohamed	30	8000.40	Directeur	1	4	Dounia	30	7000.00	Assistant	4
5	Omar	29	9000.00	Ingenieur	1	4	Dounia	30	7000.00	Assistant	4
7	Mostafa	29	9000.00	Ingenieur	NULL	4	Dounia	30	7000.00	Assistant	4

7

Programmation dans SQL

7.1 Les procédures stockées

Une procédure stockée est un programme stocké dans la base de données et callable à partir d'un client comme on peut le faire pour une requête.

Lorsque l'on crée une procédure, un des premiers soucis à est comment écrire une instruction qui se termine par un point-virgule alors que ma procédure contient un point-virgule à chaque ligne. Il faut alors changer le caractère délimiteur par défaut par un autre qui ne risque pas d'apparaître dans nos instructions, prenons par exemple le pipe '|'

```
1 DELIMITER |
```

Syntaxe générale d'une procédure

```
1 CREATE PROCEDURE name ([param[,param ...]])
```

On peut utiliser des paramètres IN, OUT ou INOUT c'est à dire en Entrée, en Sortie et à la fois en Entrée et en Sortie. Chaque paramètre est défini par son sens, son nom et son type.

Voici un premier exemple d'une procédure qui met à jour les prix en leur appliquant un coefficient

```
1 DELIMITER |
```

```
2 CREATE PROCEDURE maj_prix (IN coef FLOAT)
3 BEGIN
4 UPDATE T1 SET PRICE = PRICE * coef ;
5 END
6 |
```

```
7 DELIMITER ;
```

Chaque bloc d'instructions doit être encadré par BEGIN et END comme en Pascal, mais peut être ignoré dans le cas d'une seule instruction.

Après avoir écrit une procédure, on remet le délimiteur standard, c'est à dire le point-virgule.

```
1 DELIMITER ;
```

On lance une procédure par une commande CALL en SQL

```
1 CALL maj_prix (1.05) ;
```

Utilisation d'un paramètre de retour

Pour affecter un résultat SQL à une variable, on utilise le mot clé INTO

```
1 DELIMITER |
2 CREATE PROCEDURE get_count (OUT nb INT)
3 BEGIN
4     SELECT COUNT(*) INTO nb FROM T1;
5 END
6 |
7 DELIMITER ;
```

Pour récupérer le résultat, on doit initialiser la variable qui va recevoir le résultat.

```
1 SET @n=0;
2 CALL get_count(@n);
3 SELECT @n;
```

Pour modifier une procédure, il faut la supprimer avec DROP puis la recréer, il n'existe pas de commande ALTER PROCEDURE

```
1 DROP PROCEDURE maproc;
```

Pour afficher le code d'une procédure

```
1 SHOW CREATE PROCEDURE maproc;
```

Pour voir les procédures existantes

```
1 SHOW PROCEDURE STATUS LIKE '%%'
```

Déclaration des variables

Les variables sont déclarées par le mot clé DECLARE et les types sont les types SQL de MySQL. On peut déclarer plusieurs variables du même type sur la même ligne ou fixer une valeur d'initialisation.

```
1 DECLARE myvar CHAR(10);  
2 DECLARE i, j INT DEFAULT 0;
```

Les commentaires

```
1 /*  
2  \* une ligne  
3  \* deuxieme ligne  
4  */  
5 -- une seule ligne (il y a un espace aprs les 2 tirets)
```

Affectation directe de variables

On utilise le mot clé SET

```
1 DECLARE x INT;  
2 DECLARE nom VARCHAR(50);  
3 SET x=10;  
4 SET nom='toto';
```

Retourner des enregistrements

Si une procédure exécute une requête SELECT, les enregistrements résultants sont retournés.

```
1 DELIMITER |
2 CREATE PROCEDURE mesenreg()
3     SELECT * FROM matabale;
4 |
```

Du coup on peut l'appeler comme une requête SELECT mis à part le 'call'

Notez qu'il n'est pas possible d'appeler la procédure à partir d'une autre Requête

```
1 SELECT * FROM mesenreg(); // Ne fonctionne pas
```

7.2 Les fonctions

Une fonction comme une procédure s'exécute sur le serveur, par contre une fonction retourne un résultat et peut être utilisée directement dans une requête SQL.

Création de fonction

On utilise la commande CREATE FUNCTION name (params) RETURNS returnType

```
1 DELIMITER |
2 CREATE FUNCTION getlib (param_id INT) RETURNS CHAR(50)
3 BEGIN
4     DECLARE lib CHAR(50);
5     SELECT libelle INTO lib FROM produit WHERE id=param_id;
6     RETURN lib;
7 END;
8 |
```

Utilisation d'une fonction stockée dans une requête SQL

```
1 SELECT getlib(1) ;
```

Exemple de fonction qui arrondit un montant à 50 centimes près

```
1 DELIMITER |
2 CREATE FUNCTION arrondi50(v DECIMAL(8,2)) RETURNS DECIMAL
   (8,2)
3     RETURN ROUND((v * 2) + 0.49999)/2;
4 |
5 DELIMITER ;
```

Conditions IF THEN ELSE

```
1 IF var = 2 THEN
2     ...
3 ELSE
4     ...
5 ELSEIF
6     ...
7 END IF;
```

Conditions CASE

Suivant la valeur de la variable qui suit CASE, le programme va traiter tel ou tel cas

```
1 CASE var
2     WHEN 1 THEN ...;
3     WHEN 2 THEN ...;
4     ELSE ...; // autres cas
5 END CASE;
```

Boucles LOOP

```
1 LOOP
2     ...
3 END LOOP
```

Si on souhaite sortir de la boucle on doit rajouter une étiquette. L'appel à LEAVE suivi de l'étiquette provoque la sortie de la boucle.

```
1 LOOP
2   ...
3   IF myvar = 0 THEN LEAVE unlabel;
4 END LOOP unlabel;
```

Boucles REPEAT UNTIL

```
1 REPEAT
2   ...
3 UNTIL var = 5 END REPEAT;
```

Si on veut recommencer l'itération

```
1 un_label: REPEAT
2   ...
3   IF i = 3 THEN ITERATE un_label; END IF;
4 UNTIL i < 100 END REPEAT un_label;
```

Boucles WHILE

```
1 WHILE i < 100 DO
2   ...
3 END WHILE
```

Utilisation des curseurs (CURSOR)

Un CURSOR, c'est ce qui va nous permettre de parcourir un jeu d'enregistrements.

On doit commencer par déclarer le curseur et l'associer à une requête de type SELECT, celle qui va fournir les enregistrements. Pour cela on doit le déclarer et lui donner un nom :

```
1 DECLARE mycursor CURSOR FOR SELECT id, nom FROM matable;
```

Avant de l'exécuter, il faut déclarer des variables qui récupéreront les valeurs des champs

```
1 DECLARE var_id INT;  
2 DECLARE var_nom VARCHAR(50);
```

On doit ensuite ouvrir le curseur avec OPEN pour exécuter la requête. A chaque boucle on 'FETCH' les valeurs dans les variables :

```
1 OPEN mycursor;  
2 boucle: LOOP  
3     FETCH mycursor INTO var_id, var_nom;  
4     ...  
5     IF done THEN  
6         LEAVE boucle;  
7     END IF;  
8 END LOOP;
```

ensuite il faut fermer le curseur pour libérer les ressources

```
1 CLOSE mycursor;
```

7.3 Les triggers

Les Triggers ou déclencheurs en français, sont des procédures attachées directement à un évènement d'une table, par exemple sur chaque insertion ou chaque suppression d'enregistrement.

Un Trigger est donc rattaché à une table et à un évènement, mais on doit aussi indiquer si notre code sera déclenché avant ou après l'évènement.

```
1 CREATE TRIGGER <nomtrigger> <action_time> <event> ON <table>
```

L'action_time précise si l'action a lieu avant (BEFORE) ou après (AFTER) l'événement.

L'event c'est l'action sur laquelle on se rattache : INSERT, DELETE, UPDATE

Prenons un exemple de gestion d'une banque et créons un trigger qui calcule le solde après une opération bancaire.

```
1 CREATE DATABASE banque;
2 USE banque;
3 CREATE TABLE Client(
4     Codecli INT,
5     Nom VARCHAR(50),
6     PRIMARY KEY(Codecli)
7 );
8 --
9 CREATE TABLE Compte(
10     Codeco INT,
11     Solde INTEGER DEFAULT 0,
12     PRIMARY KEY(Codeco)
13 );
14 --
15 CREATE TABLE Transaction(
16     Codeco INT,
17     DateT DATE,
18     Montant VARCHAR(50),
19     Numero VARCHAR(50),
20     Codecli INT NOT NULL,
21     PRIMARY KEY(Codeco,Codecli,Numero),
22     FOREIGN KEY(Codeco) REFERENCES Compte(Codeco),
23     FOREIGN KEY(Codecli) REFERENCES Client(Codecli)
24 );
25 --
26 CREATE TRIGGER calcul_solde
```

```

27 AFTER INSERT
28 ON Transaction FOR EACH ROW
29     UPDATE Compte
30     SET Solde = Solde + NEW.Montant
31     WHERE Compte.Codeco = NEW.Codeco;
32 --
33 INSERT INTO Client VALUES (1,'Cheikh');
34 INSERT INTO Compte(Codeco) VALUES (2);
35 SELECT * FROM Compte;
36 INSERT INTO Transaction VALUES (2,'2022-01-1',2000,1,1);
37 INSERT INTO Transaction VALUES (2,'2022-01-1',2000,2,1);
38 INSERT INTO Transaction VALUES (2,'2024-01-1',-2000,3,1);
39 SELECT * FROM Compte;

```

Dans le trigger, on accède aux données de l'enregistrement à insérer avec le préfixe NEW

Le FOR EACH ROW est obligatoire même si dans notre cas nous ne traitons qu'un seul enregistrement.

8

Administration du serveur

☐ Création d'un utilisateur

```
1 CREATE USER 'login'@'hote' [IDENTIFIED BY 'mot_de_passe'];
```

☐ Suppression d'un utilisateur

```
1 DROP USER 'login'@'hote';
```

☐ Modification d'un utilisateur

```
1 -- Renommer un utilisateur
2 RENAME USER mama@localhost TO mamadou@localhost;
3 -- Modifier le PWD d'un utilisateur
4 SET PASSWORD FOR 'mamadou'@'localhost' = PASSWORD('toto');
```

☐ Afficher les privilèges d'un utilisateur

```
1 SHOW GRANTS FOR 'login'@'hote';
```

9

Les transactions